



DÉROULÉ DE SESSION · SUPPORT FORMATEUR

Formation Prometheus & Grafana

Session Apicil (Lyon) — jeudi, la journée pratique

Sommaire

Session Apicil (Lyon) — jeudi, la journée pratique	2
La journée en un coup d'œil	2
Ce soir (mercredi) — 25 minutes	3
8h30 — Vérification	4
9h00 — Accueil et rattrapage	4
9h15 — Les bonnes métriques et PromQL avec l'IA	5
9h40 — TP A : la boutique en quatre signaux dorés	6
11h45 — TP B : le serveur en méthode USE	8
12h15 — Auto-audit des dashboards	9
12h30 — Déjeuner	9
13h30 — Alerter sans épuiser les équipes	9
13h40 — TP C : Prometheus et Alertmanager	9
14h30 — Pause	10
15h00 — TP D : l'alerting de Grafana, et la comparaison	10
15h30 — Audit : trouvez les erreurs	11
15h45 — Thanos express	13
15h55 — Lundi matin	14

Session Apicil (Lyon) — jeudi, la journée pratique

Ce document remplace, pour le jeudi, le déroulé prévu dans « Session Apicil (Lyon) — déroulé sur 21 h ». La situation : mercredi soir, on vient à peine d'attaquer l'instrumentation (module 6). Le groupe vient de l'infrastructure, pas du développement. Je leur **donne** l'application instrumentée, et je leur ai dit que PromQL, aujourd'hui, une IA l'écrit très bien : je ne leur fais donc pas écrire de requêtes, je leur apprend à les **demandeur** et à les **vérifier**.

La journée a deux piliers, et tout le reste les sert :

1. **Les tableaux de bord** (matin) : deux dashboards construits selon les méthodes que Google et les praticiens recommandent (signaux dorés pour le service, USE pour le serveur), puis un auto-audit sur la grille des bonnes pratiques.
2. **L>alerting** (après-midi) : la même famille de besoins, traitée une fois avec Prometheus et Alertmanager, une fois avec Grafana, pour qu'ils sentent la différence dans leurs mains, puis un tableau comparatif qu'ils remplissent eux-mêmes.

Et deux séquences courtes en fin de journée : un **audit** de configuration (bonnes pratiques Prometheus et sécurité, sous forme de jeu) et **Thanos express**.

La phrase à dire à 9h00 : **aujourd'hui, vous ne codez pas, vous pilotez**.

Horaires : 9h00–16h00, déjeuner d'une heure, deux pauses d'une demi-heure, soit **cinq heures de travail effectif**, dont environ quatre les mains sur le clavier.

Supports :

- **Stagiaires** : *Guide stagiaire — Dernier jour, version pratique* (docs/stagiaire/Guide-stagiaire-Jour-3-express.pdf , et jour-3-express.md pour Notion).
- **Moi** : ce document, le deck (numéros de slides ci-dessous ; je saute tout le reste), mon Codespace de démonstration.
- **Dans le dépôt** (branche formation-2026) : rattrapage/appliquer.sh (le lab prêt), rattrapage/alerting/ (règles et Alertmanager du TP C), rattrapage/audit/ (les trois fichiers piégés), rattrapage/thanos.sh (Thanos en une commande).

La journée en un coup d'œil

Heure	Durée	Séquence	Slides	Modalité
9h00	15 min	Accueil, rattrapage en une commande		Tous ensemble
9h15	25 min	Les bonnes métriques (Google SRE, RED, USE), PromQL avec l'IA, échauffement	60, 61, 62	Exposé court puis individuel
9h40	50 min	TP A — La boutique en quatre signaux dorés (étapes 0 à 2)	66, 67	Individuel
10h30	30 min	Pause		
11h00	45 min	TP A — métier, finitions, crash test	69	Individuel
11h45	30 min	TP B — Le serveur en méthode USE, import du 1860	61	Individuel
12h15	15 min	Auto-audit des dashboards, modèle de maturité	68	Binômes
12h30	60 min	Déjeuner		
13h30	10 min	Alerter sans épuiser les équipes	82, 83, 84, 98	Exposé
13h40	50 min	TP C — Prometheus et Alertmanager	85, 86, 89	Individuel, démo au début
14h30	30 min	Pause		
15h00	30 min	TP D — L>alerting de Grafana, tableau comparatif	99, 100	Individuel puis tous ensemble
15h30	15 min	Audit : trouvez les erreurs	106, 107, 108	Binômes, jeu
15h45	10 min	Thanos express	113	Démo, suivie par qui veut
15h55	5 min	Lundi matin	117	Tous ensemble
16h00		Fin, ferme		

Ce qui est sacrifié, et je le dis à voix haute à 9h00 comme à 15h55 : écrire l'instrumentation, écrire du PromQL à la main, les recording rules (déjà en place), les droits Grafana, les notifications Teams réelles, la sauvegarde, le TP Thanos complet. Tout est dans le dépôt, avec les guides des jours 1 à 3.

Les points de contrôle. Si je suis en retard, je coupe dans cet ordre, sans remords :

1. L'étape 5 du TP A (crash test) : je la fais en démonstration.
2. Le TP B passe de 9 panels à 4 (CPU et mémoire, utilisation et saturation), plus l'import 1860.
3. La partie 5 du TP C (silence) : démonstration de 2 minutes.
4. La partie 4 du TP D (mute timing) : je la montre.
5. Thanos : démonstration seule, sans que les stagiaires le lancent.

Je ne coupe **jamais** l'audit : c'est le moment le plus vivant de l'après-midi, et c'est là que passent les bonnes pratiques de configuration et de sécurité.

Ce soir (mercredi) — 25 minutes

Pas à pas — pousser sur GitHub

Je télécharge `formation-2026-jeudi-v2.bundle` dans le dossier `.../PROMETHEUS GRAFANA/2026/`, puis dans Terminal :

```
cd "/Users/yparent/Documents/PERSO/YSYCloud/FORMATION/PROMETHEUS GRAFANA/2026/formation-observabilite-lab"
git fetch ../formation-2026-jeudi-v2.bundle formation-2026-formateur:formation-2026-formateur formation-2026:formation-2026 --
update-head-ok --force
git push --force origin formation-2026 formation-2026-formateur
```

Sur github.com, branche `formation-2026` : je dois voir `rattrapage/` avec `appliquer.sh`, `thanos.sh`, `alerting/`, `audit/`, et `docs/stagiaire/Guide-stagiaire-Jour-3-express.pdf`.

Pas à pas — tester comme un stagiaire 10 min, à faire absolument

1. github.com → le dépôt → bouton vert **Code** → onglet **Codespaces** : je rouvre mon Codespace « stagiaire » sur `formation-2026` (ou **Create codespace on formation-2026**).
2. Dans le terminal du Codespace, les trois commandes :

```
git fetch origin
git checkout origin/formation-2026 -- rattrapage/
bash rattrapage/appliquer.sh
```

3. J'attends `10 / 10` cibles UP (2 à 4 minutes la première fois).
4. Onglet **PORTS** → 3000 → Grafana, `admin / formation`. Explore → `up` → 10 lignes.
5. Je teste la commande de l'audit, qui doit répondre `FAILED ... scrape timeout greater than scrape interval` :

```
docker run --rm -v "$PWD/rattrapage/audit:/audit" --entrypoint promtool \
quay.io/prometheus/prometheus:v3.13.3 check config /audit/prometheus.yml
```

Pas à pas — préparer mon Codespace de démonstration

Mon Codespace de démo est sur `formation-2026-formateur` (avec les corrigés). Si je peux, je le passe en **4 cœurs** (github.com → Codespaces → ... → *Change machine type*) : il portera Thanos cet après-midi en plus du reste.

```
git pull --ff-only || git reset --hard origin/formation-2026-formateur
bash rattrapage/appliquer.sh
cp solutions/jour-2/dashboards/*.json grafana/dashboards/
```

La dernière ligne fait apparaître dans le dossier *Formation* de Grafana les dashboards corrigés « TP 4 - Serveur Linux » et « TP 5 - Boutique en ligne » : des exemples de résultats à montrer quand un stagiaire est perdu.

Pas à pas — le message à poster demain à 9h00

Bonjour à tous ! Pour démarrer, dans le terminal de votre Codespace :

```
git fetch origin
git checkout origin/formation-2026 -- rattrapage/
bash rattrapage/appliquer.sh
```

Attendez "10 / 10 cibles UP" (2 à 4 minutes). Le guide du jour :
docs/stagiaire/Guide-stagiaire-Jour-3-express.pdf (dans le dépôt, branche formation-2026)
Gardez aussi un onglet ouvert sur l'IA de votre choix : on va s'en servir.

8h30 — Vérification

- Codespace de démo : `./lab.sh status`, Grafana ouvert.
- Le deck en mode Présentateur, sur la slide 60.
- Onglets, dans l'ordre : le deck, Grafana (démo), Prometheus (démo), Alertmanager (démo), Inbox (démo), une IA (pour la démonstration de 9h25), le dépôt GitHub.
- Au tableau, en gros : les trois commandes du rattrapage. Et à côté, préparé pour plus tard, un tableau vide à deux colonnes « Alertmanager | Grafana » : on le remplira à 15h25.

9h00 — Accueil et rattrapage 15 min

Ce que je dis

« Bonjour à tous. On s'est arrêté hier sur l'instrumentation : une application qui expose elle-même ses métriques. Je vous propose un changement de programme. Vous êtes des gens d'infrastructure : lundi, ce n'est pas vous qui instrumenterez les applications, ce sont vos développeurs. Et PromQL, je vous l'ai dit, une IA l'écrit très bien, à condition de savoir le lui demander et de vérifier ce qu'elle rend. Donc aujourd'hui, je vous donne l'application déjà instrumentée, et on passe la journée sur ce que vous ferez vraiment : des tableaux de bord qui parlent, des alertes qui servent. Vous ne codez pas, vous pilotez. »

« Ce matin, deux tableaux de bord, construits avec les méthodes que Google recommande. Cet après-midi, les alertes, deux fois : avec Prometheus et Alertmanager, puis avec Grafana, et vous me direz lequel vous préférez et pour quoi. On finit par un jeu : un audit de configuration où il y a au moins douze erreurs à trouver. Et cinq minutes sur Thanos, pour savoir où aller quand un Prometheus ne suffit plus. »

J'envoie le message préparé dans le chat.

Pas à pas — le rattrapage, avec eux

Je le fais **en même temps qu'eux**, dans mon Codespace stagiaire projeté.

1. Terminal du Codespace (menu  → *Terminal* → *New Terminal* s'il n'est pas visible).
2. Les trois commandes. Je commente pendant que ça tourne : - « L'étape 1 sauvegarde ce que vous avez fait hier : rien n'est perdu. » - « L'étape 3 active les briques 1 à 6 : Prometheus, Grafana, l'application, le Node Exporter, les exporters, et l'Alertmanager de cet après-midi. »
- « L'étape 4 dépose le code terminé de l'application et la configuration complète de Prometheus. »
3. `10 / 10 cibles UP` : je fais lever la main de ceux qui l'ont.

Ceux qui n'ont pas 10/10 :

Symptôme	Cause probable	Remède
<code>error: pathspec 'origin/formation-2026'</code>	fetch non fait, ou mauvais dépôt	<code>git remote -v ; git fetch origin formation-2026</code> , puis refaire le checkout

Symptôme	Cause probable	Remède
Docker ne répond pas	Codespace qui vient de redémarrer	attendre 30 s, relancer le script
toomanyrequests	limite Docker Hub (Redis, Grafana)	docker login avec un compte Docker Hub gratuit, relancer
8 ou 9 cibles UP sur 10	un conteneur encore en démarrage	relancer le script : il est rejouable
port is already allocated	un binaire de mardi tourne encore	le script les arrête ; sinon fermer les vieux terminaux, relancer
Codespace supprimé		en recréer un sur formation-2026 , puis seulement bash rattrapage/appliquer.sh

Pendant que les derniers terminent, je montre le tableau « les commandes pour casser la boutique » en page 3 de leur guide : « Elles vont servir toute la journée. Un dashboard ou une alerte qu'on n'a jamais vus réagir à une panne, on ne sait pas s'ils marchent. »

9h15 — Les bonnes métriques et PromQL avec l'IA 25 min

Slide 60 — Les quatre signaux dorés 5 min

Texte complet dans les notes de la slide. L'essentiel :

« Quand on ne sait pas quoi mettre dans un tableau de bord, on ne part pas des métriques qu'on a : on part d'une méthode. La plus connue vient de Google. En 2016, Google publie gratuitement le livre qui décrit comment ses équipes d'exploitation travaillent, *Site Reliability Engineering*, sur sre.google. Chapitre 6 : si vous ne pouvez mesurer que quatre choses sur un service utilisé par des gens, mesurez la latence, le trafic, les erreurs et la saturation. »

Je détaille chacun avec la comparaison de **la voiture** : la vitesse (le trafic), le voyant moteur (les erreurs), le temps de trajet (la latence), la jauge d'essence (la saturation). « Quatre informations, et on conduit. Le reste, c'est pour le garagiste. »

Deux subtilités que Google souligne et que je répète :

- **La latence des erreurs à part.** Une erreur 500 renvoyée en 2 ms fait baisser la latence moyenne : on croit que ça va mieux, alors que ça va plus mal.
- **La saturation est le seul signal qui prévient.** Les trois autres constatent.

Slide 61 — Signaux dorés, RED, USE 3 min

« RED, c'est la version microservices : les trois premiers signaux, sans la saturation, si simple qu'on peut exiger le même dashboard pour les 200 services de l'entreprise. USE, c'est l'autre côté : pas le service, la ressource. Brendan Gregg, l'ingénieur performance de Sun puis de Netflix : pour chaque ressource, CPU, mémoire, disque, réseau, l'utilisation, la saturation, les erreurs. »

Au tableau, deux étages : en haut « le service » (signaux dorés), en bas « la machine » (USE), et une flèche de haut en bas. « Ce matin, TP A, l'étage du haut. TP B, l'étage du bas. Et un lien entre les deux. On alerte sur le haut, on diagnostique avec le bas. »

Slide 62 — PromQL : l'IA écrit, vous vérifiez 5 min

« Je vous l'ai dit : PromQL, une IA l'écrit bien. Mais elle ne connaît pas **vos** métriques, et elle se trompe avec beaucoup d'assurance. Deux compétences : bien demander, toujours vérifier. »

Pas à pas — démonstration en direct 5 min

Je fais **devant eux** une demande à l'IA, dans l'onglet préparé :

1. J'ouvre `http://...5001/metrics`, je cherche `# TYPE http_requests_total`, je copie les deux lignes `# HELP` et `# TYPE` et trois lignes de valeurs.
2. Je colle le modèle de demande de leur guide (page « PromQL avec l'IA ») en remplaçant la dernière ligne par « le taux d'erreur 5xx en pourcentage, pour toute la boutique ».
3. Je copie la réponse dans **Explore** (Grafana → Explore → Prometheus → mode *Code*). Si l'IA a mis `$_rate_interval`, je le remplace par `5m` : Explore le comprend, mais c'est une variable de dashboard.
4. Les quatre vérifications, à voix haute : ça s'exécute ; l'ordre de grandeur (0 ou presque, la boutique est saine) ; le compteur est dans un `rate` ; puis `./lab.sh chaos errors on`, et j'attends une minute que la courbe monte. « Voilà : maintenant, elle est vérifiée. » `./lab.sh chaos errors off`.

Si l'IA se trompe pendant la démonstration, **tant mieux** : c'est la meilleure démonstration possible des quatre vérifications. Les erreurs classiques : un nom de métrique inventé (`http_server_requests_seconds_count`, le nom Spring Boot), la moyenne à la place du p95, un `sum` avant le `rate`, `status="500"` au lieu de `status=~"5.."`.

La règle de sécurité, à dire lentement : « Chez un assureur, on ne colle jamais dans une IA publique des données de production : noms de clients, adresses internes, mots de passe. Ici, la boutique est fictive, on est tranquilles. Lundi, demandez à votre RSSI quel outil d'IA est autorisé. »

L'échauffement 7 min

« À vous : une requête par signal doré, dans Explore, pour toute la boutique, avec vos quatre vérifications. Notez la valeur actuelle dans votre guide. Sept minutes. »

Valeurs attendues (pour circuler et vérifier d'un coup d'œil) :

Signal	Ordre de grandeur	Erreurs d'IA fréquentes
Trafic	6 à 7 req/s	le compteur brut sans <code>rate</code> (des milliers, qui montent)
Erreurs	0 sans chaos	<code>status="500"</code> au lieu de <code>status=~"5.."</code> ; pas de division (un débit, pas un taux)
Latence p95	0,15 à 0,25 s	la moyenne <code>_sum / _count</code> ; <code>sum by (route)</code> sans <code>le</code> : « No data »
Saturation	quelques % de CPU	<code>node_cpu_seconds_total</code> sans <code>rate</code> ; oubli du <code>1 -</code> (donne le CPU libre)

9h40 — TP A : la boutique en quatre signaux dorés 50 min + 45 min

Objectif. Un dashboard de service qui répond en cinq secondes à « est-ce que ça va ? », puis montre quand, où, combien, puis ce qu'on vend. Toutes les visualisations clés : Stat, Time series, Heatmap, Pie chart, Bar gauge, Text, annotations, liens.

Slides 66 (choisir la visualisation) et 67 (ce qui rend un dashboard lisible) : je les passe en 2 minutes, puis je laisse la 66 projetée pendant le TP.

« Votre dashboard a trois étages, comme un immeuble. Au rez-de-chaussée, en haut de l'écran, la réponse à "ça va ?" : quatre cases de couleur, une par signal doré. Au premier, les mêmes signaux dans le temps, pour l'équipe technique. Au deuxième, le métier, pour le directeur commercial. Chaque panel a une question en français dans votre guide : vous la posez à l'IA, vous vérifiez, vous réglez l'affichage. Si vous bloquez plus de cinq minutes sur une requête, il y a une annexe de secours à la fin du guide. Personne ne reste bloqué sur du PromQL aujourd'hui : le sujet, c'est Grafana. »

Pas à pas — la variable (je la montre, 3 min)

1. Dashboard vide, mode édition : **Add** → **Variable** (ou **Settings** → **Variables** → **New variable**).
2. *Variable type* : Query. *Name* : `instance`. *Label* : `Instance`.
3. *Query* : `label_values(http_requests_total{job="shop-api"}, instance)`.
4. **Multi-value** et **Include All option** ; *Custom all value* : `.*`.
5. *Preview of values* : `shop-api-1:5000`, `shop-api-2:5000`. **Back to dashboard**.

« Dans chaque requête, `instance=~"$instance"`. Le `=~` parce que "All" devient `.*`, une regex. Ajoutez cette consigne à votre demande à l'IA, elle le fera. »

Pas à pas — le premier Stat, de bout en bout (je le montre, 4 min)

Le panel 1 (Trafic), projeté, en expliquant chaque clic :

1. **Add** → **Visualization**, source *Prometheus*, mode **Code**, la requête (vérifiée dans Explore).
2. En haut à droite, visualisation **Stat**.
3. Options, champ de recherche en haut : `unit` → *requests/sec (rps)*.
4. `color mode` → **Background** ; `graph mode` → **Area**.
5. **Thresholds** : pour le trafic, je choisis *Color scheme* : *Single color* bleu. « Le trafic n'est ni bon ni mauvais : il ne doit jamais être rouge. Le rouge, c'est "il faut agir". »
6. Titre **Trafic** , **Back to dashboard**, je le redimensionne en quart de largeur.

Ensuite, ils font les trois autres.

Pendant le TP : les pièges, dans l'ordre où ils arrivent

Piège	Symptôme	Ce que je dis
Unité oubliée	0.012 au lieu de 1,2 %	« Percent (0.0-1.0) si la requête donne un ratio, Percent (0-100) si elle multiplie par 100. »
Seuils à l'envers	tout est rouge	les seuils se lisent de bas en haut : <i>Base</i> vert, puis orange à 0.01, rouge à 0.05
Latence « No data »	<code>sum by (route)</code> sans <code>le</code>	« Le <code>le</code> , c'est l'escalier des buckets : sans lui, plus d'escalier. »
Heatmap grise	<i>Format</i> : <i>Heatmap</i> oublié	sous la requête : <i>Options</i> → <i>Format</i> → <i>Heatmap</i> , puis <i>Calculate from data</i> : <i>No</i>
Pie chart d'une seule part	pas de <code>by (payment_method)</code> ou légende absente	légende <code>{{payment_method}}</code>
Stock : 12 barres	pas d' <code>avg by (product)</code>	les deux instances ont chacune leur stock
Oubli de sauvegarde	tout perdu au rechargement	« Ctrl+S / Cmd+S toutes les dix minutes. »

La question du guide (étape 3) : pourquoi `rate` pour un montant ? Parce que `shop_revenue_euros_total` est un compteur : tout ce qui a été vendu depuis le démarrage de l'application, qui repart à zéro à chaque redémarrage. `rate` × 3600 donne la **vitesse**, en euros par heure, maintenant. Comparaison : le compteur kilométrique contre le compteur de vitesse.

10h25 — avant la pause

« Là où vous en êtes, sauvegardez. Après la pause : l'étage métier et les finitions qui font un dashboard professionnel. »

11h00 — la suite : l'étape 4, les finitions (je les montre, 5 min)

C'est ce qui distingue un dashboard d'amateur d'un dashboard professionnel. Je montre les trois :

1. **Le panel Text** : **Add** → *Visualization* → *Text*, mode *Markdown*, trois lignes (voir le bloc ci-dessous). Je dis : « Le premier lecteur de ce dashboard, c'est quelqu'un d'astreinte, réveillé à 3 h, qui ne l'a jamais vu. Trois lignes lui disent où il est et qui appeler. »
2. **L'annotation Chaos** (slide 69) : **Add** → *Annotation query*. Je lance `./lab.sh chaos errors on` : un trait rouge vertical sur toutes les courbes. « En post-mortem, la première question est toujours : qu'est-ce qui a changé à ce moment-là ? Vos déploiements annotés, et vous avez la réponse sans chercher. »
3. **Le lien** vers le dashboard serveur, avec *Keep time range*. « Quand le haut est rouge, un clic pour descendre d'un étage, sur la même période. »

Le texte du panel Text :

```
**Boutique en ligne : santé et ventes.**
Propriétaire : équipe Boutique · Astreinte : #boutique-astreinte
En cas d'alerte : [runbook] (https://github.com/yparent/formation-observabilite-lab)
```

Anecdote — le dashboard du directeur. On m'avait demandé « un dashboard pour le directeur ». J'en ai livré un de 40 panels, magnifique. Il a regardé, et il a demandé : « Donc, ça va ou pas ? » Depuis, la première rangée de chaque dashboard répond à cette question, en quatre chiffres colorés, et tout le reste est en dessous.

Le crash test (étape 5) : `./lab.sh traffic 30`. La réponse attendue à « est-ce une panne ? » : **non**. Le trafic et la saturation montent, le chiffre d'affaires aussi, les erreurs et la latence restent vertes. « C'est le Black Friday qui se passe bien. Si votre dashboard avait une case rouge pour le trafic, on aurait réveillé quelqu'un pour une bonne nouvelle. C'est pour ça que le trafic est en bleu. »

Le joker, pour celui qui est complètement perdu

```
git fetch origin
git checkout origin/formation-2026-formateur -- solutions/jour-2/dashboards/
cp solutions/jour-2/dashboards/*.json grafana/dashboards/
```

Dix secondes plus tard, deux dashboards corrigés sont dans le dossier *Formation* de son Grafana. Ils ne sont pas organisés en signaux dorés, mais toutes les requêtes et tous les réglages y sont.

11h45 — TP B : le serveur en méthode USE 30 min

Je remonte la slide 61 dix secondes : l'étage du bas.

« Même exercice, pour la machine. Pour chaque ressource, trois questions : est-elle occupée (utilisation) ? Est-ce que du travail attend (saturation) ? Y a-t-il des erreurs ? C'est la méthode de Brendan Gregg, et elle a un avantage énorme : elle vous empêche d'oublier une ressource. »

Les deux points à expliquer pendant le TP

La saturation CPU (panel 2) et le piège de l'IA. La charge (*load average*) compte les processus qui veulent le CPU. 2 sur une machine à 2 cœurs : pile plein. 4 : la moitié attend. D'où « charge divisée par le nombre de cœurs ». Et l'IA propose presque toujours `node_load1 / count(node_cpu_seconds_total{mode="idle"})`, qui renvoie « No data » : à gauche, une série avec les labels `instance` et `job` ; à droite, un `count` sans aucun label. PromQL apparie les séries par labels identiques : rien ne correspond. La correction : `/ on (instance) count by (instance) (...)`. « C'est le meilleur exercice de la journée : donnez le message d'erreur à l'IA, elle se corrige. Travailler avec une IA, c'est une conversation, pas un distributeur. »

La pression (PSI, panel 3). Les noyaux Linux récents mesurent directement le temps pendant lequel des tâches **attendent** une ressource : c'est la saturation à l'état pur, sans calcul. Le Node Exporter l'expose (`node_pressure_*`). « Si vous ne retenez qu'une métrique de saturation pour Linux, c'est celle-là. »

MemAvailable et pas MemFree : Linux utilise la mémoire libre comme cache disque et la rend à la demande. `MemFree` est toujours bas et fait peur pour rien.

Anecdote — le serveur à 98 % de mémoire. Des années de tickets « mémoire critique » sur un serveur de base de données, parce que la supervision regardait `MemFree`. Le serveur allait très bien : Linux faisait son travail de cache. Le jour où on est passé à `MemAvailable`, les alertes ont disparu, et personne ne les a regrettées.

La chaîne complète, à faire faire à 12h05 : `./lab.sh chaos cpu 120`. La case Saturation du dashboard Boutique passe à l'orange, un clic sur le lien, le dashboard serveur montre le CPU, la charge et la pression. « Voilà un diagnostic en deux clics. »

L'import du 1860 5 min

Dashboards → New → Import, `1860`, *Load*, source Prometheus, *Import*. (Si un proxy bloque grafana.com : télécharger le JSON sur grafana.com/grafana/dashboards et *Upload dashboard JSON file*. Dans Codespaces, pas de problème.)

« Le dashboard le plus téléchargé de grafana.com. Des dizaines de panels. Parfait pour une enquête de deux heures. À 3 h du matin, vous ouvrez le vôtre. On commence souvent par un dashboard de la communauté, et on construit son propre dashboard de synthèse par-dessus. »

12h15 — Auto-audit des dashboards 15 min

Slide 68, le modèle de maturité de Grafana (2 min, texte dans les notes), puis :

« Grille de dix critères dans votre guide. Notez votre dashboard Boutique, honnêtement. Puis échangez d'ordinateur avec votre voisin et notez le sien. Huit minutes. »

Les critères viennent de la documentation Grafana (*Dashboard best practices*) et du chapitre 6 de Google SRE. Au débrief (4 min), je fais lever les mains : « Qui a 8 ou plus ? » Puis les deux critères les plus souvent ratés, en général le 8 (annotations) et le 9 (lien vers le niveau suivant). « Ce sont les deux qui font gagner le plus de temps pendant un incident. »

Dernière question avant le déjeuner : « Chez vous, vous êtes à quel niveau de maturité ? » Réponse habituelle : faible, avec un sourire. « Lundi, vous pouvez passer à moyen en une semaine. »

12h30 — Déjeuner

« Laissez vos Codespaces ouverts. S'ils se mettent en veille, ils redémarrent avec la stack au retour : attendez 30 secondes. »

Pour moi, à 13h25 : dans mon Codespace de démo, `bash rattrapage/thanos.sh on` . Ainsi, à 15h45, le bucket aura déjà des blocs à montrer.

13h30 — Alerter sans épuiser les équipes 10 min

- **Slide 82 — Mauvaise alerte, bonne alerte.** « Google SRE, encore : on alerte sur un symptôme, pas sur une cause. Et toute alerte qui réveille quelqu'un doit être urgente et actionnable. Si la réponse à "qu'est-ce que je fais quand ça sonne ?" est "je regarde", ce n'est pas une alerte, c'est un panel. »
- **Slide 83 — 400 notifications par jour.** L'anecdote de la fatigue d'alerte (notes de la slide).
- **Slide 84 — Le budget d'erreur.** Je le présente comme « le niveau suivant », sans le pratiquer : « On ne dit plus "plus de 5 % d'erreurs", on dit "au rythme actuel, on va rater notre promesse du mois". C'est comme ça qu'on parle aux métiers. » Le détail est dans les notes.
- **Slide 98 — Alertmanager ↔ Grafana.** La correspondance des concepts. « Cet après-midi, vous allez faire la même famille de choses deux fois, avec deux outils. Retenez ce tableau : les concepts sont les mêmes, seuls les noms changent. La différence est ailleurs, et c'est vous qui allez me la dire à 15h25. »

13h40 — TP C : Prometheus et Alertmanager 50 min

Objectif. Comprendre les deux étages : Prometheus **évalue** les règles, l'Alertmanager **décide** qui prévenir, comment, quand. Lire des règles, prédire un routage, voir l'inhibition et le silence fonctionner.

Les fichiers sont **fournis** dans `rattrapage/alerting/` : ils les copient, les lisent, les modifient un peu. Personne n'écrit 80 lignes de YAML.

Partie 1 — Les règles (10 min) · slide 85

Je projette `rattrapage/alerting/alerts.yml` et je lis `ShopHighErrorRate` à voix haute :

« Une règle, c'est quatre choses. Une requête qui renvoie quelque chose quand ça va mal : le tamis, tout ce qui dépasse 5 % passe. Un `for` : ça doit durer une minute, sinon on ignore. Des labels : `severity` et `team`, c'est l'adresse sur l'enveloppe, l'Alertmanager ne lit que ça. Des annotations : le texte de la lettre, avec le lien vers le runbook. »

Correction des questions du guide :

1. Symptômes : `ShopHighErrorRate`, `ShopCheckoutSlow`, `BlackboxProbeFailed`, et `TargetDown` (à la limite : c'est l'absence de mesure). Cause : `HostHighCpuLoad`, et c'est pour ça qu'elle est en `warning`.
2. `for` : l'anti-faux positif, et le passage *Pending* → *Firing* (slide 86). `team` : sert **uniquement** au routage.
3. Réveille : `ShopHighErrorRate` (critical). Ne doit jamais réveiller : `HostHighCpuLoad`.

Partie 2 — L'arbre de routage (15 min) · slide 89

Je dessine l'arbre **au tableau** à partir de `amtool config routes` :

```
racine → inbox-default
├── severity=critical → astreinte-teams (continue: true)
├── team=boutique → boutique-slack
└── team=infra → infra-inbox
```

« L'alerte entre par la racine et descend. Elle s'arrête à la première branche qui lui correspond, sauf si la branche dit `continue: true` : elle continue alors de descendre. C'est un aiguillage de gare. Le `continue`, c'est la photocopie : l'astreinte reçoit l'original, l'équipe reçoit une copie. »

Correction des prédictions :

Labels	Receivers
severity=critical team=boutique	astreinte-teams , boutique-slack
severity=warning team=boutique	boutique-slack
severity=warning team=infra	infra-inbox
severity=critical (sans team)	astreinte-teams
severity=info team=logistique	inbox-default (aucune branche : la racine)

Sans `continue: true`, la première ligne ne donne plus que `astreinte-teams` : l'équipe boutique ne sait pas que son site est en panne. « C'est l'erreur de routage la plus fréquente en production. »

La commande de test, qu'ils garderont : `amtool config routes test`. « On teste son routage **avant** la panne, pas pendant. »

Partie 3 — Casser et suivre 10 min

`./lab.sh chaos errors on`. Chronologie attendue : *Pending* en 15 à 30 s ; *Firing* une minute après (le `for`) ; Inbox 10 s plus tard (`group_wait`). **Deux messages par groupe** : un en Teams (astreinte), un en Slack (boutique), à cause du `continue`. Les deux instances sont dans le **même** message : `group_by: [alertname, job]`. « Le regroupement : deux instances en panne, une seule notification. Avec 200 serveurs, c'est ce qui vous sauve la nuit. »

Le *resolved* arrive jusqu'à une minute après la fin du chaos (`group_interval: 1m` ; en production, c'est souvent 5 minutes).

Partie 4 — L'inhibition 10 min

« Le site est en erreur, l'astreinte est déjà réveillée. Est-ce qu'elle a besoin, en plus, d'une alerte "le paiement est lent" ? Non : c'est du bruit. L'inhibition, c'est "si A sonne, fais taire B". »

Ils ajoutent le bloc de six lignes, `check`, `reload`, puis `chaos errors et latency`. Au bout de deux minutes : `ShopCheckoutSlow` existe, mais apparaît **Inhibited** (ou *suppressed* dans `amtool alert query --inhibited`), et rien n'est arrivé dans l'Inbox pour elle.

Partie 5 — Le silence 5 min

Démonstration rapide dans l'interface de l'Alertmanager (bouton **New Silence**), matcher `team="boutique"`, 30 minutes, commentaire « Maintenance base de données ». « La différence avec l'inhibition : l'inhibition est une règle permanente dans la configuration ; le silence est ponctuel, posé par un humain, avec son nom et une raison, pour une durée. »

À 14h28 : « `./lab.sh chaos reset`, et pause. »

14h30 — Pause 30 min

15h00 — TP D : l'alerting de Grafana, et la comparaison 30 min

Slide 99 (quand utiliser quoi), 1 minute, puis :

« Le directeur commercial ne lira jamais un fichier YAML, et il veut pouvoir changer son seuil lui-même. On va lui faire son alerte dans Grafana. Et on ne refait **pas** les alertes techniques du TP C dans Grafana : la même alerte des deux côtés, c'est deux notifications, deux vérités, et un jour l'une des deux est fausse. »

Pas à pas — le premier contact point, projeté 2 min

Alerting → Notification configuration → Contact points → + Create contact point, nom `equipe-commerce`, *Webhook*, URL `http://inbox:8080/webhook/commerce`, **Test**, **Save**.

« Pourquoi `inbox` et pas `localhost` ? C'est Grafana qui envoie, depuis son conteneur. Pour lui, `localhost`, c'est lui-même. Les conteneurs s'appellent par leur nom. »

Ensuite, ils font le reste seuls avec le guide.

Pendant le TP : les pièges

Piège	Ce que je dis
La règle reste en <i>Normal</i>	le seuil : <i>IS BELOW</i> la moitié de la valeur actuelle (environ 170 000 €/h dans le lab), et <code>./lab.sh traffic 1</code>
<i>Preview</i> montre des courbes au lieu d'une valeur	requête en <i>Range</i> : passer en Instant
La notification part vers <code>astreinte-grafana</code>	le label <code>team=commerce</code> manque sur la règle, ou le matcher de la politique est mal saisi
Rien dans l'Inbox après <i>Firing</i>	<i>Group wait</i> par défaut de 30 s, plus l'intervalle d'évaluation : patience, 1 à 2 minutes

Le déclenchement : `./lab.sh traffic 1`. Le chiffre d'affaires tombe d'environ 170 000 à 20 000 €/h en deux minutes, la règle passe *Pending* puis *Firing*, le message arrive dans l'Inbox sur le canal `commerce`. `./lab.sh traffic 6`.

Le tableau comparatif, tous ensemble 7 min

Je remplis au tableau le tableau préparé le matin, **avec leurs réponses** :

	Prometheus + Alertmanager	Grafana
Où vit la configuration ?	des fichiers YAML	la base de Grafana (ou des fichiers de provisioning)
Versionner, relire	Git, merge request, naturellement	export YAML, provisioning, Git Sync : possible, moins naturel
Tester avant de déployer	<code>promtool test rules</code> , <code>amtool config routes test</code>	<i>Preview</i> dans l'interface
Qui modifie ?	les équipes qui touchent au YAML	n'importe qui avec les droits sur le dossier
Plusieurs sources (SQL, logs)	non, métriques Prometheus seulement	oui, toute source de données Grafana
Si Grafana tombe	les alertes continuent	plus d'alertes
Inhibition	oui	pas d'équivalent direct

La conclusion, à dire : « Les alertes techniques, critiques, qui doivent survivre à tout : Prometheus et Alertmanager, dans Git, testées. Les alertes métier, multi-sources, ou pour des équipes qui vivent dans Grafana : Grafana. Les deux coexistent très bien, chacun sur son terrain. »

15h30 — Audit : trouvez les erreurs 15 min

Juste avant : « Lancez `bash rattrapage/thanos.sh on`, il démarrera pendant l'audit. » (Si le Codespace est lent ou si le temps manque, on saute : je montrerai Thanos sur mon écran.)

Les slides 106 et 107, en 3 minutes

Je les passe **vite** : « Voilà les règles. Maintenant, on va voir si vous savez les appliquer. » Le texte complet est dans les notes.

Slide 108 — le jeu

« Une équipe vous confie sa supervision. Elle vous dit : elle marche. C'est vrai, elle démarre. Trois fichiers dans `rattrapage/audit/` . Par binôme, huit minutes, trouvez tout ce qui ne va pas. Un point par erreur, un point de plus si vous proposez la correction. Il y en a au moins douze. »

Chronomètre au tableau. À 8 minutes : la commande `promtool` de leur guide. Il ne trouve **qu'une** erreur, le `scrape_timeout` . « Voilà la différence entre un outil et vous : `promtool` vérifie la syntaxe, pas le bon sens. »

Puis tour de table : une erreur par binôme, à tour de rôle, jusqu'à épuisement. Je coche dans la liste ci-dessous et je complète à la fin ce qui n'a pas été trouvé.

La liste complète

`docker-compose.yml`

#	Erreur	Correction
1	<code>prom/prometheus:latest</code> , <code>grafana/grafana:latest</code> : versions non figées (et Docker Hub, limite de pulls)	version exacte, la LTS : <code>quay.io/prometheus/prometheus:v3.13.3</code> , <code>grafana/grafana:13.2.1</code>
2	<code>0.0.0.0:9090</code> et <code>0.0.0.0:3000</code> : exposés sur toutes les interfaces, sans authentification	réseau interne ou <code>127.0.0.1</code> , reverse proxy avec SSO, <code>--web.config.file</code> (TLS, basic auth bcrypt)
3	<code>--storage.tsdb.retention.time=3y</code> en local	15 à 30 jours ; l'historique long, c'est Thanos ou Mimir
4	<code>--web.enable-admin-api</code> ouvert à tous : n'importe qui peut supprimer des séries	le retirer, ou le protéger
5	<code>--web.enable-lifecycle</code> sans authentification : n'importe qui peut arrêter Prometheus (<code>/-/quit</code>)	seulement derrière une authentification
6	Aucun volume pour <code>/prometheus</code> : toutes les données perdues au premier redémarrage	un volume nommé, comme dans le lab
7	Grafana sans volume : dashboards et utilisateurs perdus	volume <code>/var/lib/grafana</code> , et tout en provisioning
8	<code>GF_SECURITY_ADMIN_PASSWORD=admin</code> : le mot de passe par défaut, en clair	un secret, via <code>GF_SECURITY_ADMIN_PASSWORD__FILE</code>
9	Accès anonyme activé, en rôle Admin : n'importe qui administre Grafana	<code>GF_AUTH_ANONYMOUS_ENABLED=false</code> , SSO

`prometheus.yml`

#	Erreur	Correction
10	<code>scrape_timeout: 30s</code> supérieur à <code>scrape_interval: 15s</code> : Prometheus refuse de démarrer (la seule que <code>promtool</code> trouve)	timeout inférieur à l'intervalle, 10 s par exemple
11	Pas d' <code>external_labels</code> (<code>cluster</code> , <code>replica</code>)	les ajouter : indispensable pour la HA, Thanos, la fédération
12	Pas de bloc <code>alerting</code> : les règles s'évaluent, mais aucune alerte ne part nulle part	<code>alerting: alertmanagers: ...</code>
13	<code>scrape_interval: 1s</code> sur <code>app</code> : 15 fois plus d'échantillons, pour rien	15 s
14	<code>honor_labels: true</code> sur une application : la cible peut écraser <code>job</code> et <code>instance</code>	réservé à la Pushgateway et à la fédération
15	Mot de passe en clair dans le YAML (qui finira dans Git)	<code>password_file</code>
16	<code>insecure_skip_verify: true</code> : TLS sans vérification du certificat	<code>ca_file</code> avec l'autorité de l'entreprise

#	Erreur	Correction
17	Pas de <code>sample_limit</code> sur un exporter tiers	<code>sample_limit: 10000</code> par exemple
18	Job <code>test2</code> : nom sans sens, adresses IP en dur, aucun label <code>env</code> / <code>team</code>	un nom parlant, du DNS ou <code>file_sd_configs</code> , des labels de routage

alerts.yml

#	Erreur	Correction
19	CPU : un compteur (<code>node_cpu_seconds_total</code>) comparé brut, sans <code>rate</code> : toujours vrai	<code>rate(...)</code> , et en %
20	Une cause (le CPU) en <code>critical</code> : on réveille quelqu'un pour un serveur qui travaille	<code>warning</code> , ou mieux, alerter sur le symptôme (latence, erreurs)
21	valeur: " <code>{{ \$value }}</code> " en <code>label</code> : une nouvelle alerte à chaque évaluation, qui clignote et inonde	la valeur va dans une annotation
22	Aucun <code>for</code> : la moindre pointe d'une seconde déclenche	<code>for: 5m</code>
23	Aucune annotation : ni résumé, ni runbook	<code>summary</code> , <code>description</code> , <code>runbook_url</code>
24	<code>ErreursBoutique</code> : <code>> 0</code> alerte dès la première erreur, sur <code>status="500"</code> seulement, sans ratio, une alerte par route et par méthode	le ratio 5xx par instance <code>> 0.05</code> , comme au TP C
25	Pas de tests de règles	<code>promtool test rules</code> en CI

Anecdote — le Grafana public. Des chercheurs en sécurité trouvent régulièrement, sur Internet, des milliers de Grafana ouverts avec `admin/admin` et des Prometheus sans authentification qui listent le nom de chaque machine interne d'une entreprise. Une métrique, c'est une cartographie de votre système d'information offerte à qui la demande.

Le binôme gagnant : applaudissements. C'est bête, et ça marche.

15h45 — Thanos express 10 min

Slide 113, sur mon Codespace de démo où Thanos tourne depuis 13h25.

« Un Prometheus garde quelques semaines, sur un seul serveur. Trois problèmes arrivent un jour : je veux un an d'historique ; j'ai plusieurs Prometheus, un par site, et je veux une vue globale ; je veux deux Prometheus jumeaux pour la haute disponibilité, sans voir tout en double. Thanos répond aux trois, sans toucher aux Prometheus : on leur colle un **sidecar**. »

Pas à pas — la démonstration 5 min

- Port **10902**, **Stores** : deux sidecars (`replica=prom-1`, `replica=prom-2`) et le Store Gateway. « Deux Prometheus qui scrapent les mêmes cibles, comme deux caméras qui filment la même scène. »
- `up{job="shop-api"}` : **2** séries. Je décoche **Use Deduplication** : **4**. « Le Querier sait que `replica` distingue les jumeaux : il garde une seule copie. C'est pour ça qu'on a mis des `external_labels` dès mardi, et que l'audit de tout à l'heure en demandait. »
- Le bucket, puisque Thanos tourne depuis deux heures : `docker compose exec thanos-store ls /bucket` : des dossiers au nom étrange, un par bloc de 10 minutes. « En production, c'est un bucket S3 ou OVH Object Storage : quelques euros par mois pour des années d'historique. »
- Grafana → Explore → source **Thanos** : la même requête. « Vos dashboards peuvent pointer sur Thanos sans rien changer d'autre. »

Ceux qui l'ont lancé refont les étapes 1, 2 et 4 chez eux, puis `bash rattrapage/thanos.sh off`.

« Les alternatives : Grafana Mimir et VictoriaMetrics, qui remplacent le stockage au lieu de le compléter. Même API : Grafana ne voit pas la différence. Le TP complet est dans le guide du jour 3. »

15h55 — Lundi matin 5 min

Slide 117.

« Lundi matin, ne faites pas tout. Choisissez **un** service. Un seul. Faites-le scraper, donnez-lui ses quatre signaux dorés en haut d'un dashboard, et **une** alerte sur un symptôme que vos utilisateurs sentiraient, avec un responsable et un runbook. C'est tout. Dans un mois, vous aurez dix services, parce que vos collègues viendront vous demander le même. »

Tour de table éclair : un service chacun, à voix haute (« Mon service : ... » dans leur guide). Ça engage.

« Mardi matin, vous ne saviez pas ce qu'était une série temporelle. Aujourd'hui, vous avez construit deux tableaux de bord selon les méthodes de Google, monté deux systèmes d'alerte et trouvé vingt erreurs dans la configuration d'une autre équipe. Les sources sont en dernière page de votre guide, le dépôt reste ouvert. Merci à tous. »

16h00 : fin.